

A good structure for this milestone is to make it production-ready from the beginning so later RAG, embeddings, APIs, and UI can plug in without major refactoring.

Recommended Repository Structure

```
manual-rag-system/
|
├─ app/
|   ├── __init__.py
|   ├── parser.py
|   ├── chunker.py
|   ├── embeddings.py
|   ├── vectordb.py
|   ├── retrieval.py
|   ├── config.py
|   └─ utils.py
|
├─ data/
|   ├── raw/
|   |   └─ sample_manual.pdf
|   ├── parsed/
|   |   └─ sample_manual.txt
|   └─ processed/
|
├─ notebooks/
|
├─ tests/
|   ├── test_parser.py
|   └─ test_chunking.py
|
├─ scripts/
|   └─ parse_pdf.py
|
├─ .gitignore
├─ README.md
├─ requirements.txt
└─ LICENSE
```



requirements.txt

```
llama-parse
llama-index
pymupdf
python-dotenv
tqdm
pandas
numpy
```



```
pathlib
pytest
```

Optional later:

```
sentence-transformers
chromadb
faiss-cpu
qdrant-client
langchain
openai
```



parser.py

This parser first tries LlamaParse.

If unavailable, it automatically falls back to PyMuPDF.

</> Python



▶ Run

```
from pathlib import Path
import fitz

try:
    from llama_parse import LlamaParse
    LLAMA_AVAILABLE = True
except:
    LLAMA_AVAILABLE = False

class PDFParser:

    def __init__(self, api_key=None):
        self.api_key = api_key

    def parse(self, pdf_path):

        if LLAMA_AVAILABLE and self.api_key:
            return self._llamaparse(pdf_path)

        return self._pymupdf(pdf_path)

    def _llamaparse(self, pdf_path):

        parser = LlamaParse(
            api_key=self.api_key,
            result_type="text"
        )
```

```

docs = parser.load_data(pdf_path)

text = ""

for doc in docs:
    text += doc.text + "\n"

return text

def _pymupdf(self, pdf_path):

    doc = fitz.open(pdf_path)

    pages = []

    for page in doc:
        pages.append(page.get_text())

    return "\n".join(pages)

if __name__ == "__main__":

    pdf = "data/raw/sample_manual.pdf"

    parser = PDFParser()

    text = parser.parse(pdf)

    output = Path("data/parsed/sample_manual.txt")

    output.parent.mkdir(exist_ok=True)

    output.write_text(text, encoding="utf-8")

    print("PDF Parsed Successfully")

```

parse_pdf.py

</> Python



▶ Run

```

from app.parser import PDFParser
from pathlib import Path

PDF = "data/raw/sample_manual.pdf"

parser = PDFParser()

text = parser.parse(PDF)

```

```
Path("data/parsed").mkdir(exist_ok=True)

Path("data/parsed/sample_manual.txt").write_text(
    text,
    encoding="utf-8"
)

print("Done.")
```

.gitignore

⌵ gitignore



```
__pycache__/  
*.pyc  
.env  
.idea  
.vscode  
  
data/processed/*  
data/parsed/*  
!data/parsed/.gitkeep  
  
*.ipynb_checkpoints
```

README.md

⌵ Markdown



```
# Manual RAG Parser  
  
A PDF parsing module for technical manuals.  
  
## Features  
  
- LlamaParse support  
- Automatic PyMuPDF fallback  
- Clean text extraction  
- Ready for RAG pipelines  
  
## Install  
  
pip install -r requirements.txt  
  
## Usage
```

```
python scripts/parse_pdf.py
```

Expected Output

After running:

</> Bash



```
python scripts/parse_pdf.py
```

You should get:

data/



```
|— raw/
|   sample_manual.pdf
|
|— parsed/
|   sample_manual.txt
```

The extracted text will look like:

SECTION 1



Safety Information

Always disconnect power before servicing.

Motor Rotation

Check phase sequence before startup.

Electrical Wiring

Use copper conductors only.

...

Initial Git Commit

</> Bash



```
git init

git add .

git commit -m "Initial project setup with PDF parser"

git branch -M main

git remote add origin https://github.com/<username>/manual-rag-system.git

git push -u origin main
```

Next Steps

Once this skeleton is in place, the next logical milestone is to implement:

1. Text cleaning (remove headers, footers, page numbers, OCR artifacts).
2. Intelligent chunking (by headings and token limits rather than fixed page boundaries).
3. Embedding generation (e.g., with Sentence Transformers or OpenAI embeddings).
4. Vector database integration (e.g., ChromaDB or FAISS) to prepare for the RAG retrieval pipeline.